1) Implementation of a heart rate monitor

Group No. B2
Date 22/05/25

Last Name/First Name	ID Number
Miotto Isabella	2066715
Orso Marco	2074059
Sette Andrea	2066747

Purpose of the exercise: to make a heart rate monitor and pulse oximeter based on infrared LED, red LED, photodiode, analog signal conditioning circuit, microcontroller board and display.

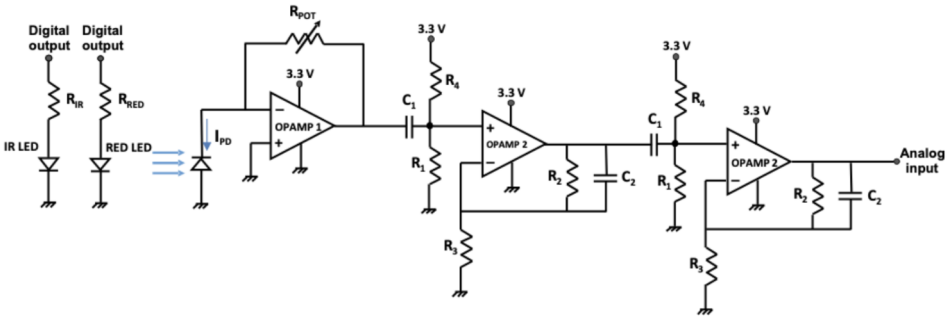
Instrumentation needed:

- Microcontroller board
- Breadboard
- Display
- Electronic components, see table below

Components needed:

Component type	Manufacturer Code/Value
Infrared LED, λ=850 nm SMT	Osram Opto, SFH 4250S
Red LED, λ=630 nm SMT	Osram Opto, LH T674
IR Photodiode/Visible SMT	Vishay VBP104S
Rail-to-rail operational amplifier	MCP6002
Resistances	To be calculated
Capacitor C1	10 µF
Capacitor C2	1 µF
Potentiometer RPOT	100 KΩ
TFT display	HX8357
Microcontroller board	
Breadboard and cables	

The circuit is powered through the USB port of the pc.



Circuit Schematic

1.1) Prelab

1. Define the values of the resistors R_{IR} and R_{RED} so that currents equal to $I_{RED} = 15\ [mA]$ and $I_{IR} = 3\ [mA]$, respectively, are obtained on the two red and infrared LEDs (use high current pins on microcontroller board, or add transistors to control the current)

$$R_{RED} = 833\ [\Omega]$$
$$R_{IR} = 1\ [k\Omega]$$

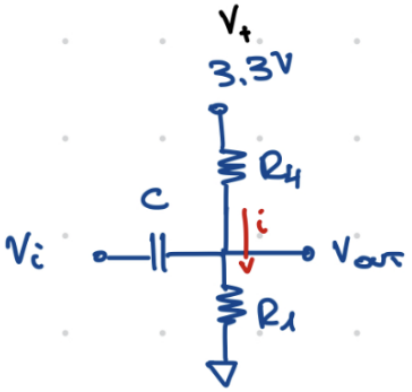
2. Calculate the transfer function of the high-pass filter consisting of C_1, R_1, R_4

By looking at the picture in the AC analysis we can define

$$R = R_1 // R_4 = \frac{R_1 R_4}{R_1 + R_4}$$

Also the current has the following value

$$i = \frac{V_i}{R + \frac{1}{j\omega C_1}}$$



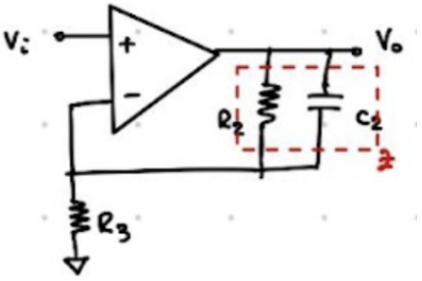
Circuit

$$V_{out} = \frac{V_i}{R + \frac{1}{j\omega C_1}} R = \frac{j\omega C_1 V_i}{1 + j\omega C_1 R} R \rightarrow G'(j\omega) = \frac{j\omega C_1 R}{1 + j\omega C_1 R}$$

3. Calculate the transfer function of the low-pass filter consisting of R_2, R_3, C_2

By looking at the picture we can define

$$Z = R_2 // \frac{1}{j\omega C_2} = \frac{R_2}{1 + j\omega C_2 R_2}$$



Circuit

The non inverting op-amp configuration has the following transfer function

$$G''(j\omega) = 1 + \frac{Z}{R_3} = \ldots = \frac{R_2 + R_3}{R_3} \frac{1 + j\omega C_2 \frac{R_2 R_3}{R_2 + R_3}}{1 + j\omega C_2 R_2}$$

4. Calculate the overall transfer function of each bandpass filter
The overall transfer function is just the product of the previous two:

$$G(j\omega) = G'(j\omega)G''(j\omega) = \ldots = \frac{R_2 + R_3}{R_3} \frac{(j\omega C_1 R)(1 + j\omega C_2 \frac{R_2 R_3}{R_2 + R_3})}{(1 + j\omega C_1 R)(1 + j\omega C_2 R_2)}$$

5. Define the values of the resistors R_1, R_2, R_3, R_4 to obtain

- That the high-pass filter consisting of C_1, R_1, R_4 has a cut-off frequency of $0.8\ [Hz]$
- That the rest dc voltage at the terminals v_+ is equal to $0.15\ [V]$
  The values of the capacitors and cut-off frequency is already given, therefore we can use the, to calculate the value of R:

  $$F_1 = 0.8 = \frac{1}{2\pi C_2 R} \rightarrow R = \frac{1}{2\pi C_2 F} \approx 20\ [k\Omega]$$

  Now, by noticing that $V_+ = V_{R1} =$ we can find

  $$3.3\frac{R_1}{R_1 + R_4} = 0.15 \rightarrow R_1 = \frac{1}{21} R_4 \rightarrow \begin{cases} R = \frac{R_1 R_4}{R_1 + R_4} = 20\ [k\Omega] \\ R_1 = \frac{1}{21} R_4 \end{cases} \rightarrow \begin{cases} R_1 \approx 21 & [k\Omega] \\ R_4 \approx 440 & [k\Omega] \end{cases}$$

- That the low-pass filter consisting of R_2, R_3, C_2 has a cutoff frequency of $3\ [Hz]$
- That the dc gain of the low-pass filter consisting of R_2, R_3, C_2 is equal to $11\ [V]$
  As before

  $$F_2 = \frac{1}{2\pi C_2 R_2} \rightarrow R_2 = 5.3\ [k\Omega]$$

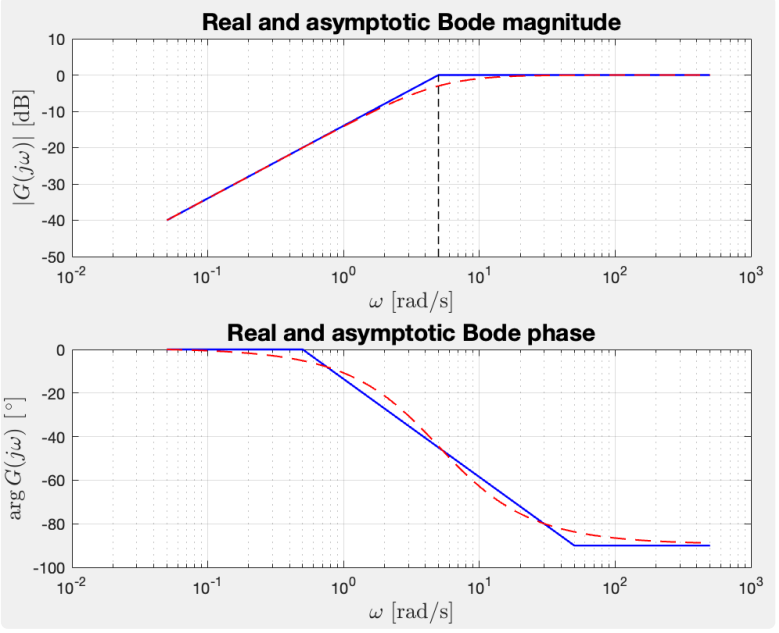
  And from the dc gain:

  $$G = 11 = 1 + \frac{R_2}{R_3} \rightarrow R_3 = 53\ [k\Omega]$$

Choose the nearest commercial value for the resistors R_1, R_2, R_3, R_4

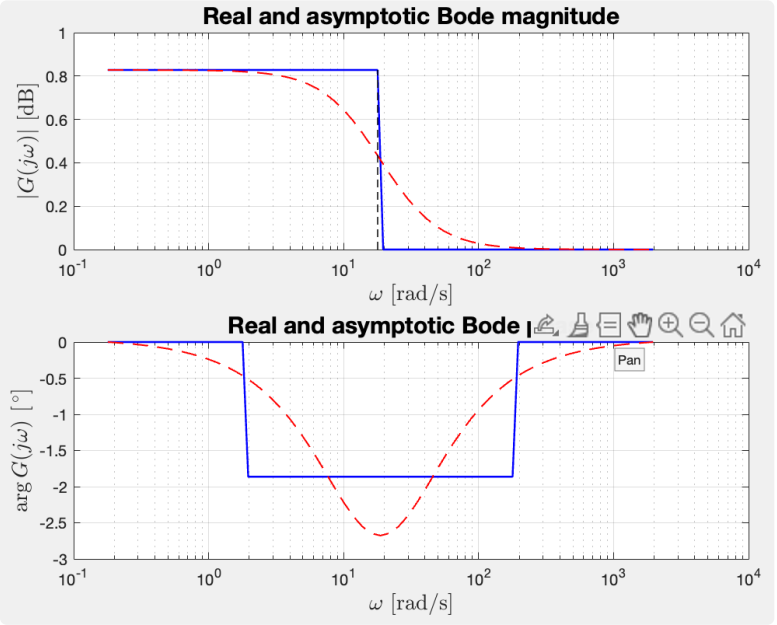
$$R_1 = 21,\ R_2 = 5.6,\ R_3 = 56,\ R_4 = 440\ [k\Omega]$$

2. With the values chosen in step 6, plot the frequency response
- of the high-pass filter consisting of C1, R1, R4



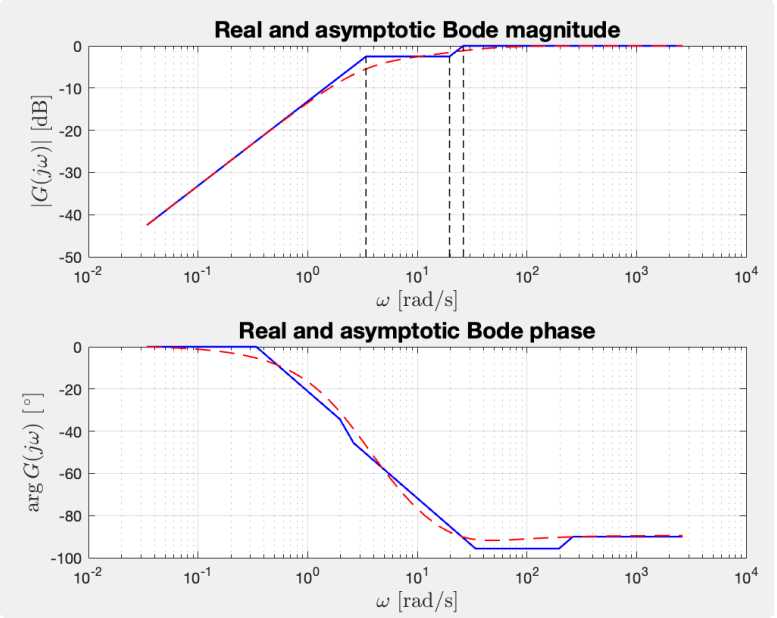
G'(s)

- of the low-pass filter consisting of R2, R3, C2. (Although Matlab didn't collaborate well with the asymptotic plot)



G"(s)

- of (one of) the bandpass filters



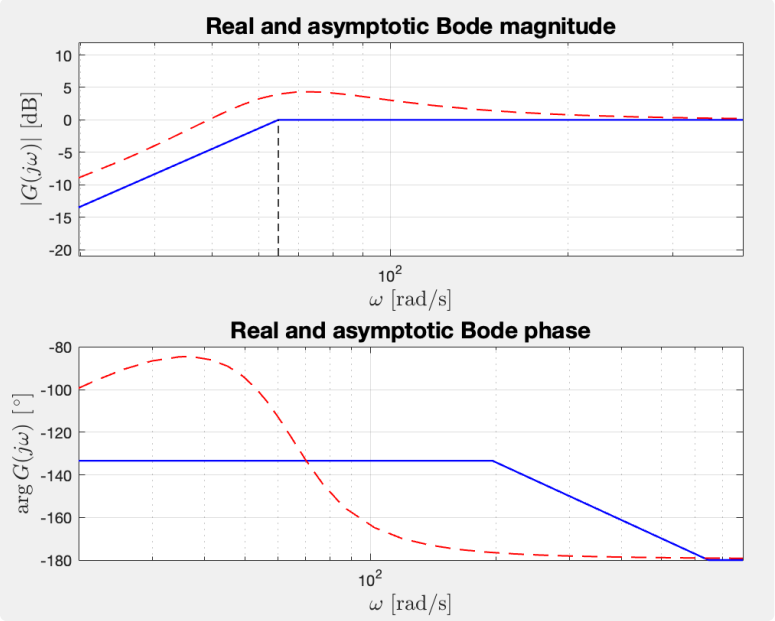
G(s)

Add to the report the cut-off frequencies obtained using the commercial values of the resistors (which will be different from 0.8 Hz and 3 Hz)

$$F_1 = 0.794,\ F_2 = 2.84\ [Hz]$$

We suppose that the pass band filter $G'(s) \cdot G''(s)$ isn't well shown by the bode diagram since the two cutoff frequencies are really close.

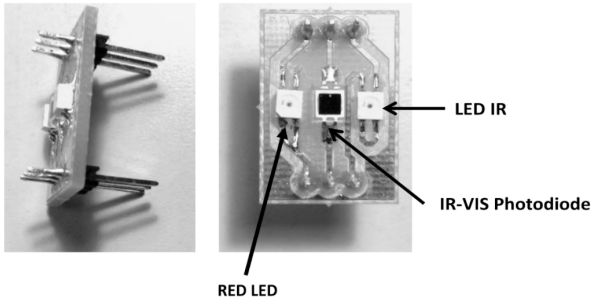
In fact, by analyzing the final output $G(s) \cdot G(s)$ we end up with a pass band that is displayed more clearly



Band Pass

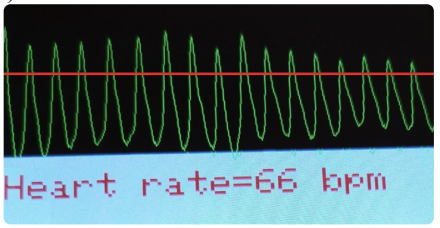
1.2) In the Laboratory

1. Mount the circuit show below, and connect the V_o signal to one of the analog inputs of the microcontroller board
   Use the custom board containing VIS/IR photodiode and IR LED diode shown in the figure below. Refer to the component datasheets to define the polarity of the components



Board

2. Write a code to capture the signal at the output of the heart rate monitor, graphing it on the TFT display at the same time (see example in figure). Use the infrared LED for the time being for this analysis
   Hints: acquire a value every 20-30 ms, place it in an array (equal in size to the width of the screen), use the function tft.drawLine(x-1, y(x-1), x, y(x), color) to draw the segment corresponding to the last pair of data acquired. The input values to the ADC (0 to 1023) should be re-scaled so that they can all be represented on the display
3. Once the Array of values has been acquired, stop the acquisition. Write a function that will re-scale the array so that it is displayed on the top half of the screen, along with any other writing (see example in figure); report the corresponding code below
   Hint: divide the code into functions as follows:
   -function findmax(int data[]): calculates the maximum value (in y) of the array data[]
   -function findmin(int data[]): calculates the minimum value (in y) of the array data[]
   -function rescalearray(int data[]): rescales the array data[] (via the map function) so that it occupies half the screen
   -function plotarray(int data[]): draws the plot of the array data[] on the display
4. **Optional:** From the acquired array, write a code to measure the heart rate and display the corresponding value on the display (see example in figure)



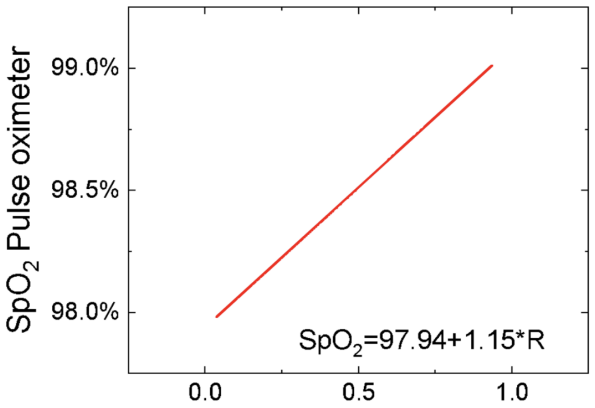
Figure

5. **Optional:** Write the code to measure the oxygen concentration in the blood by evaluating the ratio R between the absorption of the red and infrared component, according to the following formula

$$R = \frac{\left(\frac{AC}{DC}\right)_{RED}}{\left(\frac{AC}{DC}\right)_{IR}} = \frac{\frac{V_{max,RED} - V_{min,RED}}{V_{min,RED}}}{\frac{V_{max,IR} - V_{min.IR}}{V_{min,IR}}}$$

From R, the saturation level can be calculated, by using the formula below.

$$SpO_2 = 97.94 + 1.15 \cdot R$$



Graph

```
#define HX8357_BLACK   0x0000
#define HX8357_BLUE    0x001F
#define HX8357_RED     0xF800
#define HX8357_GREEN   0x07E0
#define HX8357_CYAN    0x07FF
#define HX8357_MAGENTA 0xF81F
#define HX8357_YELLOW  0xFFE0
#define HX8357_WHITE   0xFFFF


#include <Adafruit_GFX.h>
#include <Adafruit_HX8357.h>
#include <SPI.h>


#define TFT_CS 10
#define TFT_DC 9
#define TFT_RST 8

Adafruit_HX8357 tft = Adafruit_HX8357(TFT_CS, TFT_DC, TFT_RST);

#define ANALOG_PIN A0
#define RED_PIN 5
#define IR_PIN 6
#define SCREEN_WIDTH 480
#define SCREEN_HEIGHT 320
#define TOP_HALF_HEIGHT (SCREEN_HEIGHT / 2)


int data[SCREEN_WIDTH];
int red[SCREEN_WIDTH];
int ir[SCREEN_WIDTH];
unsigned long startTime;

int count=0;
int dataCount=0;
int oldMean;

const int samples = 480;
```

```
const float samplingTime = 25;
const int calibrationThresh = 1005; //adjust as needed

bool setupComplete = false;

const unsigned char epd_bitmap_free_heart_icon_3510_thumb [] PROGMEM = {
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x07, 0x80,
0x00, 0x3c, 0x00,
    0x00, 0x00, 0x3f, 0xf0, 0x01, 0xff, 0x80, 0x00, 0x00, 0xf8, 0x78, 0x03, 0xc3,
0xe0, 0x00, 0x01,
    0xc0, 0x1e, 0x0f, 0x00, 0x70, 0x00, 0x03, 0x80, 0x07, 0x1c, 0x00, 0x38, 0x00,
0x07, 0x00, 0x03,
    0xb8, 0x00, 0x1c, 0x00, 0x0e, 0x00, 0x01, 0xf0, 0x00, 0x0e, 0x00, 0x0c, 0x00,
0x00, 0xe0, 0x00,
    0x06, 0x00, 0x0c, 0x00, 0x00, 0x40, 0x00, 0x06, 0x00, 0x18, 0x00, 0x00, 0x00,
0x00, 0x03, 0x00,
    0x18, 0x00, 0x00, 0x00, 0x00, 0x03, 0x00, 0x18, 0x00, 0x00, 0x00, 0x00, 0x03,
0x00, 0x18, 0x00,
    0x00, 0x00, 0x00, 0x03, 0x00, 0x18, 0x00, 0x00, 0x00, 0x00, 0x03, 0x00, 0x18,
0x00, 0x00, 0x00,
    0x00, 0x03, 0x00, 0x0c, 0x00, 0x00, 0x00, 0x00, 0x06, 0x00, 0x0c, 0x00, 0x00,
0x00, 0x00, 0x06,
    0x00, 0x06, 0x00, 0x00, 0x00, 0x00, 0x0c, 0x00, 0x07, 0x00, 0x00, 0x00, 0x00,
0x1c, 0x00, 0x03,
    0x80, 0x00, 0x00, 0x00, 0x38, 0x00, 0x01, 0xc0, 0x00, 0x00, 0x00, 0x70, 0x00,
0x00, 0xe0, 0x00,
    0x00, 0x00, 0xe0, 0x00, 0x00, 0x70, 0x00, 0x00, 0x01, 0xc0, 0x00, 0x00, 0x38,
0x00, 0x00, 0x03,
    0x80, 0x00, 0x00, 0x1c, 0x00, 0x00, 0x07, 0x00, 0x00, 0x00, 0x0e, 0x00, 0x00,
0x0e, 0x00, 0x00,
    0x00, 0x07, 0x00, 0x00, 0x1c, 0x00, 0x00, 0x00, 0x03, 0x80, 0x00, 0x38, 0x00,
0x00, 0x00, 0x01,
    0xc0, 0x00, 0x70, 0x00, 0x00, 0x00, 0x00, 0xe0, 0x00, 0xe0, 0x00, 0x00, 0x00,
0x00, 0x70, 0x01,
    0xc0, 0x00, 0x00, 0x00, 0x00, 0x38, 0x03, 0x80, 0x00, 0x00, 0x00, 0x00, 0x1c,
0x07, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x0e, 0x0e, 0x00, 0x00, 0x00, 0x00, 0x00, 0x07, 0x1c, 0x00,
0x00, 0x00, 0x00,
    0x00, 0x03, 0xb8, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0xf0, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
    0xe0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00
};

const int epd_bitmap_allArray_LEN = 1;
const unsigned char* epd_bitmap_allArray[1] = {
    epd_bitmap_free_heart_icon_3510_thumb
};

void setup() {
  Serial.begin(9600);

  tft.begin();
  tft.setRotation(3);
  tft.fillScreen(HX8357_BLACK);

  pinMode(5, OUTPUT);
  digitalWrite(5, HIGH);
  pinMode(6, OUTPUT);
  digitalWrite(6, HIGH);

  const char* text = "HEART MONITOR";
  displayText(text, 159, HX8357_MAGENTA);
```

```
  const char* text2 = "Calibrating...";
  tft.fillRect(0, 265, SCREEN_WIDTH, 55, HX8357_BLACK);
  displayText(text2, 265, HX8357_WHITE);

  tft.drawBitmap(50, 159, epd_bitmap_allArray[0], 51, 51, HX8357_WHITE);
}


void loop() {
  delay((int)samplingTime);

  // Lettura IR
  digitalWrite(RED_PIN, LOW);
  digitalWrite(IR_PIN, HIGH);
  delayMicroseconds(100);
  int irVal = analogRead(ANALOG_PIN);
  ir[count] = irVal;

  // Lettura RED
  digitalWrite(IR_PIN, LOW);
  digitalWrite(RED_PIN, HIGH);
  delayMicroseconds(100);
  int redVal = analogRead(ANALOG_PIN);
  red[count] = redVal;


  data[count] = redVal;

  plotData(data);

  if ((count % 50 == 0 || count == 0) && setupComplete)
    plotBPM();

  count++;
  if (count >= SCREEN_WIDTH) {
    count = 0;
  }

  //Logic for getting valid samples only once a finger is put
  dataCount++;
  if (dataCount >= SCREEN_WIDTH) setupComplete = true;

  if (findRecentMax(data, 2) > calibrationThresh) {
    setupComplete = false;
    dataCount = 0;

    const char* text2 = "Calibrating...";
    tft.fillRect(0, 265, SCREEN_WIDTH, 55, HX8357_BLACK);
    displayText(text2, 265, HX8357_WHITE);
    tft.fillRect(0, 0, SCREEN_WIDTH, 107, HX8357_BLACK);
  }

  if (count % 240 == 0 && setupComplete) {
    float spo2 = calculateSpO2(red, ir, SCREEN_WIDTH);
    char text[30];
    sprintf(text, "Saturation = %.1f%%", spo2);
    tft.fillRect(0, 215, SCREEN_WIDTH, 40, HX8357_BLACK);
    displayText(text, 215, HX8357_WHITE);
  }
}

//PLOT
void plotData(int data[]) {
  int sampleCount = 100;
  int maxVal = findRecentMax(data, sampleCount);
  int minVal = findRecentMin(data, sampleCount);

  //Erease old screen data
  int eraseStart = (count + 1) % SCREEN_WIDTH;
  int eraseWidth = 50;

  if (eraseStart + eraseWidth <= SCREEN_WIDTH) {
```

```
    tft.fillRect(eraseStart, 0, eraseWidth, 107, HX8357_BLACK);
  } else {
    int firstPart = SCREEN_WIDTH - eraseStart;
    int secondPart = eraseWidth - firstPart;

    tft.fillRect(eraseStart, 0, firstPart, 107, HX8357_BLACK);
    tft.fillRect(0, 0, secondPart, 107, HX8357_BLACK);
  }

  //actual plot
  if (count > 0) {
    int y1 = map(data[count - 1], minVal, maxVal, 106, 0);
    int y2 = map(data[count],     minVal, maxVal, 106, 0);

    tft.drawLine(count - 1, y1, count, y2, HX8357_RED);
  }

  // Draw mean line in white
  long sum = 0;
  for (int i = 0; i <= count; i++) {
    sum += data[i];
  }
  int mean = sum / (count + 1);
  int meanY = map(mean, minVal, maxVal, 106, 0);
  tft.drawLine(count-1, oldMean, count, meanY, HX8357_WHITE);
  oldMean = meanY;
}

void plotBPM() {
  const char* text2 = "BPM = ";
  char fullText[20];
  sprintf(fullText, "%s%d", text2, getBPM(data));
  tft.fillRect(0, 265, SCREEN_WIDTH, 55, HX8357_BLACK);
  displayText(fullText, 265, HX8357_WHITE);
}

//CALCULATIONS
int getBPM(int arr[]) {
  int beatCount = 0;
  bool aboveUpper = false;
  bool belowLower = true;  // Start assuming we're below lower threshold
  int lastBeat = -1000;

  int minVal = findMin(arr);
  int maxVal = findMax(arr);

  int upperThreshold = minVal + ((maxVal - minVal) * 65) / 100;
  int lowerThreshold = minVal + ((maxVal - minVal) * 35) / 100;

  //use these for the mean value
  //upperThreshold = (minVal+maxVal)/2;
  //lowerThreshold = upperThreshold;

  for (int i = 0; i < SCREEN_WIDTH; i++) {
    if (arr[i] > upperThreshold && belowLower) {
      beatCount++;
      lastBeat = i;
      aboveUpper = true;
      belowLower = false;
      Serial.print("Beat at sample "); Serial.println(i);
    }
    else if (arr[i] < lowerThreshold) {
      belowLower = true;
      aboveUpper = false;
    }
  }

  float seconds = (float)(samples * (samplingTime / 1000.0f));
  float bpm = (beatCount * 60.0f) / seconds;
  return (int)((bpm/1)+0.5f); //maybe bpm/2
}
```

```
float calculateSpO2(int red[], int ir[], int size) {
  int redMax = red[0];
  int redMin = red[0];
  int irMax  = ir[0];
  int irMin  = ir[0];

  for (int i = 1; i < size; i++) {
    if (red[i] > redMax) redMax = red[i];
    if (red[i] < redMin) redMin = red[i];
    if (ir[i] > irMax)   irMax  = ir[i];
    if (ir[i] < irMin)   irMin  = ir[i];
  }

  float acRed = (float)(redMax - redMin);
  float dcRed = (float)(redMin);
  float acIr  = (float)(irMax - irMin);
  float dcIr  = (float)(irMin);

  if (dcRed == 0 || dcIr == 0 || acIr == 0) return 0.0;

  float r = (acRed / dcRed) / (acIr / dcIr);

  float spo2 = 97.94 + (1.15 * r);
  return spo2;
}

//MIN/MAX
int findMax(int arr[]) {
 int maxVal = arr[0];
 for (int i = 1; i < SCREEN_WIDTH; i++) {
   if (arr[i] > maxVal) maxVal = arr[i];
 }
 return maxVal;
}

int findMin(int arr[]) {
 int minVal = arr[0];
 for (int i = 1; i < SCREEN_WIDTH; i++) {
   if (arr[i] < minVal) minVal = arr[i];
 }
 return minVal;
}

//These functions are used to better scale the plot
int findRecentMin(int arr[], int N) {
  int start = max(0, count - N);
  int minVal = arr[start];
  for (int i = start + 1; i <= count; i++) {
    if (arr[i] < minVal) minVal = arr[i];
  }
  return minVal;
}

int findRecentMax(int arr[], int N) {
  int start = max(0, count - N);
  int maxVal = arr[start];
  for (int i = start + 1; i <= count; i++) {
    if (arr[i] > maxVal) maxVal = arr[i];
  }
  return maxVal;
}

//TEXT
void displayText(const char* text, int y, int color) {
  tft.setTextSize(3);
  tft.setTextColor(color);

  int16_t x = getX(text);

  tft.setCursor(x, y);
  tft.print(text);
}
```

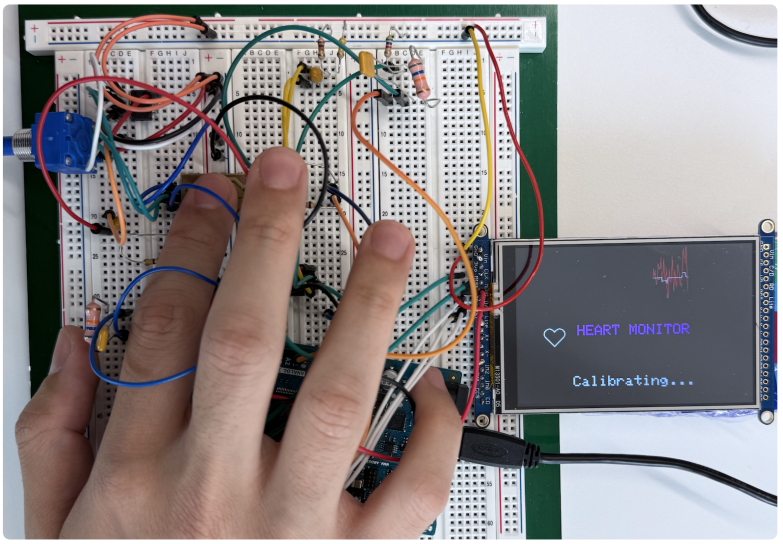
```
int16_t getX(const char* fullStr){
  int16_t x1, y1;
  uint16_t w, h;
  tft.getTextBounds(fullStr, 0, 0, &x1, &y1, &w, &h);

  return (tft.width() - w) / 2;
}
```

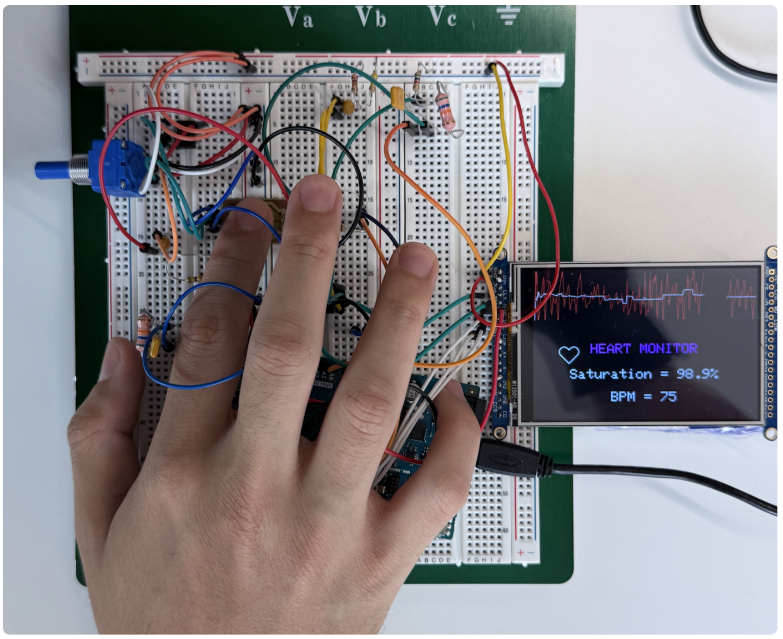
We only provide the final code implementing all 4 functions
Here is a quick rundown of the code and it's functions:

- *setup():* Here we setup all devices, in particular the text for "HEART MONITOR" and the related image are only set here
- *min/max:* here we have 2 functions for getting the min/max values of the array. The "recent" version of the min/max is used to better scale the heart rate plot and to detect if a finger was put on the sensor. the global min/max functions are used to find the min/max of the entire array. This function will be called only once enough valid samples are recorded in the circular array
- *getBPM():* To better detect the heartbeat we implemented a function with different upper and lower thresholds to avoid to count some beats twice due to noise inflicted oscillations. The sampling time and sample count result in quantization steps of 5.
- *plotData():* this function plots the recorded data in the region with y$\in [0-106]$ of the screen. It is scaled based on the min/max values of the last 100 samples to have it always occupy the most of the reserved screen area. Moreover it deletes the samples displayed in front of it (even circularly) to always have the oldest and newest data displayed. The mean is calculated every frame and is drawn by calling drawline() with the new mean and old mean.
- *loop():* Here we do a few different things
  - we flicker the sensors to be able to measure SpO_2 data.
  - we save the data and increase the counter
  - we also implemented a calibration logic: It is possible to detect when a finger is set on the sensor by looking at the measure of a peak maximum voltage at around 1010 pwm value. Then we added a counter that gets reset every time a finger is sensed in order to display the BPM only once enough samples of said person are recorded in the circular array. Once enough samples are saved the text changes from "calibrating..." to the correct information. This then gets updated every 50 samples for the bpm and 240 for the SpO_2. The old data displayed is removed.

Here are some images of the final results:



Calibration Just Started



Enough Recorded Data